

1. 论文名称: Seven Failure Points When Engineering a Retrieval Augmented Generation System

Abstract

- **研究主题:** 检索增强生成 (RAG) 系统在软件工程中的应用。
- **RAG系统目标:**
 1. **减少虚构响应:** 降低大型语言模型生成的虚构响应。
 2. **关联源与参考:** 将回答与其来源的文档或参考资料关联。
 3. **消除注释需求:** 去除为文档添加元数据注释的必要性。
- **RAG系统局限性:**
 - 受限于信息检索系统的固有限制。
 - 过度依赖大型语言模型。
- **研究内容:** 通过三个不同领域的案例研究分析RAG系统的失败点。
- **案例研究领域:**
 1. 研究
 2. 教育
 3. 生物医学
- **失败点总结:** 设计RAG系统时需考虑的7个失败点。
- **关键启示:**
 1. **系统验证时机:** RAG系统的验证只有在运行期间才是可行的。
 2. **系统稳健性发展:** RAG系统的稳健性是随着时间逐步发展的，而非一开始就完全设计出来。
- **研究方向:** 提出针对软件工程社区的RAG系统研究的潜在方向。

1 Introduction

- **引言概述:**
 - **大型语言模型 (LLMs) 的进步:** 包括ChatGPT，为软件工程师提供了新能力。
 - **LLMs的局限性:** 在处理最新知识或特定领域知识时存在局限，尤其是那些储存在公司仓库中的。
 - **解决方案:**
 1. **微调LLMs:** 使用特定领域文档继续训练LLMs。
 2. **使用检索增强生成 (RAG) 系统:** 依赖LLMs生成答案，使用现有的 (可扩展的) 知识工件。
- **RAG系统简介:**
 - **定义:** RAG系统是一个将信息检索能力与LLMs的生成能力相结合的系统。

- **组件：**
 1. 检索组件：从数据存储中检索用户查询相关信息。
 2. 生成组件：使用检索到的信息作为生成用户查询答案的上下文。
- **重要性：**所有非结构化信息现在都可以被索引和查询，减少了开发时间和知识图谱创建的需求。
- **研究内容：**
 - **案例研究：**通过三个案例研究展示了RAG系统的实施所面临的挑战。
 - **研究目的：**
 1. 给从业者提供参考。
 2. 提供RAG系统的研究路线图。
- **研究问题：**
 - **失败点：**在设计RAG系统时会出现哪些失败点？
 - **关键考量：**在工程化RAG系统时需要考虑哪些关键点？
- **研究贡献：**
 - 提供RAG系统中出现的失败点目录。
 - 提供三个案例研究的经验报告。
 - 基于三个案例研究得出的教训，为RAG系统提供研究方向。
- **实验和分析：**
 - 使用BioASQ数据集进行了实验，报告潜在的失败点。
 - 实验涉及15,000个文档和1,000个问题与答案对。
 - 使用GPT-4生成响应，然后通过OpenAI evals¹进行验证。
 - 手动检查，分析所有不一致和被标记为错误的标签，以识别模式。
- **案例研究的当前状态：**两个案例研究目前在迪肯大学（Deakin University）运行。

2 Related work

- **相关工作：**
 - **检索增强生成（RAG）的应用：**利用文档增强大型语言模型的预训练和推理。
 - **优势：**由于计算成本和数据准备时间，不经训练或微调就使用RAG是一种有吸引力的提议。
 - **挑战：**在使用大型语言模型进行信息提取，如长文本处理时，会面临挑战。
- **调研情况：**
 - 最近的一项调研显示大型语言模型在RAG流程中的多个环节（检索器、数据生成、改写器、reader）中被使用。
 - 本工作从软件工程的角度补充了这项调研，聚焦于实践者面临的问题和必要的软件工程研究。
- **研究缺口：**

- 尽管有关于RAG系统的基准测试工作，但尚未有研究关注在实施过程中出现的失败。
- 软件工程研究已经探索了RAG系统在代码相关任务中的使用，但RAG系统的应用范围更广。
- **挑战和错误：**
 - RAG系统在实施过程中出现的错误和失败与其他信息检索系统相似，包括：
 1. 没有用于查询重写的指标。
 2. 文档重新排序问题。
 3. 有效内容总结。
 - 结果证实了这些问题，并指出了使用大型语言模型的语义和生成特性相关的独特方面，包括评估事实准确性。

3 检索增强生成（RAG）

- **背景：**随着像ChatGPT这样的大型语言模型服务日益普及，它们已被用于问答系统。
- **挑战：**
 1. 幻觉（hallucinations）：LLM产生看似正确但实际错误的回答。
 2. 不受限制（unbounded）：无法直接或更新输出内容，除非通过提示工程。
- **RAG系统定义：**RAG是一种旨在克服直接使用LLM限制的信息检索方法。
- **工作流程：**
 - 将自然语言查询转换为嵌入（embedding），用于语义搜索文档集。
 - 然后将检索到的文档传递给大型语言模型以生成答案。
 - RAG系统概述包括两个独立过程：索引和查询。

3.1 索引过程

- **检索系统：**使用嵌入（embeddings）来提供文档的压缩语义表示。
- **嵌入：**表示为数字向量。
- **文档处理：**每个文档被分割成小块，每块通过嵌入模型转换为嵌入，然后在数据库中索引。
- **设计决策：**
 1. 如何分块文档以及块应该有多大。
 2. 如果块太小，可能无法回答某些问题；如果块太长，则回答可能包含生成的噪声。
- **不同文档的处理：**
 - 需要不同的分块和处理阶段。
 - 例如，视频内容需要转录流程提取音频并在编码前转换为文本。
- **嵌入选择：**
 - 选择哪种嵌入也很重要，因为改变嵌入策略需要重新索引所有块。
 - 应根据能否语义地检索正确答案来选择嵌入。

- 这个过程取决于块的大小、预期的问题类型、内容和应用领域的结构。

3.2 查询过程

- **运行时发生：**将自然语言问题转换为一般查询，然后转换成嵌入以在数据库中定位相关文档。
- **检索文档：**使用相似性方法（如余弦相似性）检索Top-k相似文档。
- **文档重排：**为了增加找到答案的块位于顶部的可能性，检索到的文档将被重新排名。
- **合并器（Consolidator）：**负责处理块，以克服大型语言模型的令牌限制和速率限制。
- **RAG系统设计考虑：**
 - 考虑到文本量的限制和包含块数量的限制。
 - 考虑到在线服务对于一定时间内使用的tokens数量的限制，这影响了系统的响应时间。
- **RAG管道的最终阶段：**
 - **答案提取：**从生成的文本中提取答案。
 - **读取器职责：**
 1. 过滤提示中的噪声。
 2. 遵循格式化指令（例如，将答案作为选项列表）。
 3. 生成查询返回的输出。
- **RAG系统实现：**
 - 需要定制多个提示来处理问题和答案。
 - 确保返回与领域相关的问题。
- **RAG系统的新领域：**
 - 使用大型语言模型回答实时问题从文档中开启了新的应用领域。
 - RAG系统是新能力，因为没有现成数据，需要通过实验发现，可以通过：
 1. 合成数据生成。
 2. 最小测试的系统试点。

4 案例研究

4.1 Cognitive Reviewer

是一个为研究者分析科学文件而设计的 RAG 系统。研究者可以指定一个研究问题或目标，然后上传相关的研究论文。系统将根据研究者的目标对文档进行排序，以便手动审查。

4.2 AI Tutor

是一个 RAG 系统，学生可以询问有关课程的问题，答案则来源于学习内容。学生可以通过访问源列表来核实答案的来源。AI Tutor 通过集成到 Deakin 大学的学习管理系统来工作，索引所有内容，包括 PDF 文档、视频和文本文档。

4.3 Biomedical Question and Answer

是为了在更大规模上探索问题，创建了一个使用 BioASQ 数据集的 RAG 系统。该数据集包含问题、链接到文档和答案，答案可以是 yes/no、文本摘要、事实或列表。这个数据集由生物医学专家准备，并包含特定领域的问题和答案对。

5 RAG的失败点

FP1 - Missing Content:

当可用文档中没有答案时，RAG 系统无法回答问题。理想情况下，系统应该回答“对不起，我不知道”。

FP2 - Missed the Top Ranked Documents:

即使文档中有答案，但是没有被足够高地排名以被返回给用户。

FP3 - Not in Context - Consolidation Strategy Limitations:

答案虽然被检索出来，但没有被包含在生成答案的上下文中。这通常发生在从数据库返回许多文档并且进行了整合过程以检索答案时。

FP4 - Not Extracted:

答案虽然在上下文中，但大型语言模型没有提取出正确的答案。这通常发生在上下文中存在太多噪声或矛盾信息时。

FP5 - Wrong Format:

问题要求以特定格式提取信息（如表格或列表），但是大型语言模型忽略了指令。

FP6 - Incorrect Specificity:

返回的答案要么不够具体，要么过于具体，无法满足用户的需求。这可能发生在 RAG 系统设计者对某个问题期望有特定结果时，比如教师为学生提供答案。在这种情况下，答案应该提供具体的教育内容，而不仅仅是答案。当用户不确定如何问问题并且问题过于泛泛时，也会发生具体性不正确的情况。

FP7 - Incomplete:

答案不是不正确，但是遗漏了一些信息，即使这些信息在上下文中可以提取。例如，一个问题可能是“文件 A、B 和 C 中涵盖的关键点是什么？”更好的方法是单独提问这些问题。

6 经验教训和未来研究方向

6.1 分块和嵌入 (Chunking and Embeddings)

- **分块**：看似简单，但对检索过程有重大影响，尤其是分块的质量会影响到与用户查询的匹配。
- **嵌入**：是活跃的研究领域，包括为多媒体和多模态块生成嵌入，如表格、图形和公式。
- **未来研究**：
 1. 探索分块方法及其对下游过程（如嵌入和相似性匹配）的影响。
 2. 比较不同分块技术的评估框架。

6.2 RAG与微调 (RAG vs Finetuning)

- **LLMs**：由于训练数据量大，且在发布前已应用微调任务，因而是出色的世界模型。
- **微调**：需要定制内部数据集来适应和训练LLM，但所有数据都被集成到模型中，这可能涉及隐私/安全问题。
- **RAG**：提供实用解决方案，允许根据需要分块数据，只使用相关块的上下文来请求LLM生成答案。
- **未来研究**：
 1. 比较微调和RAG范式，包括准确性、延迟、运营成本和稳健性等因素。

6.3 测试和监控RAG系统

- **软件工程实践**：RAG系统的软件工程最佳实践仍在发展中。
- **测试和监控**：需要适当的测试数据和质量度量来帮助工程师进行质量权衡。
- **未来研究**：
 1. 探索适用于RAG系统的自适应系统理念以支持监控和调整。
 2. 对于RAG系统的测试数据生成和质量度量标准进行研究。

7 结论

- **RAG系统**：一种利用大型语言模型（LLMs）的新型信息检索系统。
- **论文内容**：
 - 提出了从三个案例研究中学到的经验教训。
 - 包括一项涉及15,000份文档和1,000个问题的实证调查。
 - 提出了实施RAG系统时面临的挑战。
 - 提出了未来研究方向，涉及：
 1. 分块和嵌入。
 2. RAG与微调。
 3. 测试和监控。

- **大型语言模型的未来：**
 - 将继续获得工程师和研究人员感兴趣的新能力。
 - 论文提供了从软件工程视角对RAG系统的首次调查。